Promo Finder: ranked promos from a microdrama screenplay

Problem 1. Reads the screenplays of a show and returns ranked 20 to 60 second promos: episode, start and end of every segment, hook, cliffhanger, one line on why, and a rank. Rerunnable on a new show with any OpenAI-compatible model.

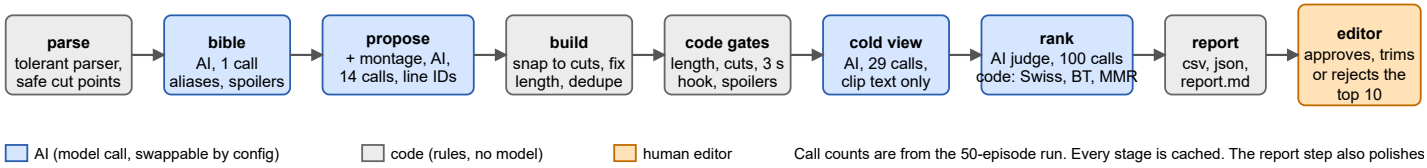
Why Problem 1

It is the step where an editor's time grows with every new show (watch every episode, pick moments by hand, new shows every week), and the output can be checked by rules: a clip has a length, a hook within 3 seconds, a clean cut and an unresolved ending. The input pack gives 50 real episodes to run on, and a tool that can be rerun on a new show is what the interview asks for.

Assumptions

- Promos are judged from the **script text only**, no video, audio or picture. "Makes sense on its own" means a cold reader can say who the people are, what is at stake and what is next.
- **Episodes 1 to 10 are the free episodes**: a hook from them gets +0.05 on its score (config `free_eps`). English, US audience. A clip is one stretch or up to three segments, never across a clean-cut boundary that falls inside a spoken line.
- Where the brief was silent I chose the simplest option and logged it (`docs/assumptions.md`). Clips are sent to Gemini and Groq; free tiers may use traffic for training, so a confidential show should use a paid or private endpoint.
- No hand-labelled answers exist yet, so precision against an editor is **not measured**.

How it works: AI, code, and where the editor approves



Models only return **line IDs**. Code turns IDs into seconds and enforces 20 to 60 s, the hook within 3 s of the start, cuts that fall inside no dialogue, and segment order. The cold view and the judge see only the clip text (no episode numbers, no scores), and the judge answers hook, ending and clarity separately in both orders to cancel position bias. Models are YAML profiles with limits, fallbacks and a response cache; `--profile-map` swaps one at run time, and an offline keyword mode needs no API. The editor stays in the loop: the tool returns a ranked list with transcripts and the editor approves the top 10 (no review screen is built, the editor works from `promos.csv` and `report.md`).

Cost and scale

Measured on the 50-episode pack (103 minutes): **147 model calls, 426k tokens (390k in, 36k out), about 12 minutes, $0** on free tiers. At paid Gemini prices (ai.google.dev/gemini-api/docs/pricing, checked 9 Nov 2026) that is **$0.21 per show** with every step on Gemini 3.5 Flash-Lite ($0.30 in, $2.50 out per 1M tokens), **$0.32** with Gemini 3.6 Flash on the three long-context steps at its promotional $0.75 / $3.75 (to 31 Dec 2026), and **$0.52** at its list price $1.50 / $7.50. Multiply by shows released: at 10 shows a week that is about $2 to $5 a week, at 50 shows about $10 to $26. (I do not know the real weekly count, so the cost is per show times that number.) Shows are independent, so they run in parallel; the limit on free tiers is requests (5 to 15 per minute, 20 to 500 per day per model), not tokens, and a paid key lifts it. Changing one prompt reruns only the stages after it. The editor reviews about 10 clips a show instead of watching 50 episodes.

How I know it is good, and where it is not

The tool's top 10 against baselines, each compared pair by pair by an LLM judge in both orders (win = 1, split decision = 0.5; 10 pairs each, so one pair is 0.1). From `outputs/duke/eval.json`.

Baseline	Tool win rate	Split
random 45 s windows	0.85	0.1
last 45 s of an episode	0.75	0.3
offline keyword heuristic	0.70	0.4
one prompt over the whole show	0.65	0.1

Perturbation: original preferred	early	late
gemini-3.5-flash-lite (structured)	0.35	0.36
gpt-oss-120b (structured)	0.25	0.44
gemini-3.5-flash-lite (first judge)	0.35	0.44

The tool beats the baselines on this judge but the judge fails its own check: given a clip and a copy with the hook buried 8 s earlier or the answer shown, it should prefer the original more than 80% of the time and prefers it 25 to 44%, with both models and both prompts. So the judge's ranking cannot be trusted to punish a buried hook or a late cut, and the win rates measure agreement with an LLM, not with an editor. Position bias fell from 39.6% to 20.8% of pairs after splitting the judge into three questions. Swapping the proposing and judging models keeps only 2 of the top 10. Honest limits: no hand labels, text only, 10 pairs per row.

Where it picks badly (`docs/failures.md`, found with code checks and the tool's own ratings, not by watching):

- **Answer inside the clip:** 3 promos contained the line that answers their own question and the cold view still called them open-ended. A final code step now cuts the end before that line (2 fixed) or flags it G8 and sends it to the bottom (1). It only knows the bible's 5 questions, so other early answers are not caught.
- **Spoilers:** 7 promos show a line the bible marked as a twist, but the bible lists only 7 such lines for 50 episodes and the gate is a soft penalty.
- **Near-duplicates:** pairs just under the build thresholds (0.5 time, 0.6 words) slipped through; the final step uses 0.4 and 0.5 and dropped 2. Those numbers are a judgement call.
- **Mid-sentence starts:** 3 of the top 30 opened on the tail of a sentence; the final step moved 2 to open on the hook line, the third is not fixed.
- **Weak hooks and no payoff link:** 4 promos with a hook rating of 2, and 25 of 71 with no link to a question the show later answers (ranks 5, 9, 14, 15, 16 are in the top 16).
- **Gaps:** the model proposed nothing for episodes 6 and 8; 3 of the top 10 end exactly where an episode ends (its own built-in cliffhanger).

AI tools used

Claude Code (Claude Sonnet 5.5) wrote the code and tests phase by phase from a written spec, and a separate Claude session (Opus) wrote the plan and reviewed every phase. The tool itself calls **Gemini 3.6 Flash, Gemini 3 Flash, Gemini 3.5 Flash-Lite and 3.1 Flash-Lite** (free tier) and **Groq gpt-oss-120b**. **One place they got it wrong:** the assistant reported that the tool beat every baseline with a 0.60 win rate against the one-prompt baseline, but `eval.json` said 0.4444. The baseline is itself a model reply, an invalid first reply is never cached, and the second eval run asked again and got 9 clips instead of 10. The reviewer caught the mismatch; the baseline is now saved and reused, and all numbers here are read from `eval.json`. Other misses: a model that never answered (gemini-3.8-flash, about 19 minutes lost to retries), and a first proposal prompt that gave 97 s spans and only 3 proposals for 5 episodes.